Air Quality Monitoring System

Microproject Report

Submitted by:
Gouri Ajith(MGP24EVLSI014)
Lakshmipriya K G(MGP24EVLSI017)
Semester 3,Electronics Engineering (VLSI Design And Technology)
Saintgits College of Engineering

Air Quality Monitoring System

1. Introduction and Objectives

Air pollution has emerged as one of the most critical environmental and public health challenges. Exposure to harmful pollutants can cause severe respiratory illnesses. This project aims to monitor indoor or outdoor air quality in real-time, using an ESP32 microcontroller and an MQ-135 gas sensor. The system processes the gas concentration readings and sends live data to a cloud dashboard (ThingSpeak / Blynk) for real-time remote monitoring.

Objectives:

- Measure and monitor air quality (PPM of harmful gases) in real time.
- Send data to a cloud dashboard (ThingSpeak / Blynk) via Wi-Fi.
- Support low-cost, scalable, and remote environmental wellness tracking.

2. Block Diagram and Explanation

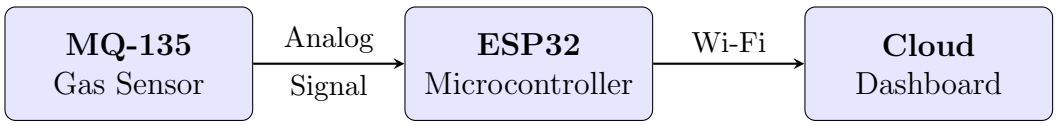


Figure 1: Block Diagram of Air Quality Monitoring System

Explanation: The MQ-135 gas sensor detects harmful gases (CO_2, NH_3, benzene, smoke, NO_x) and sends an analog signal to the ESP32. The ESP32 processes and transmits the data to the cloud server through its built-in Wi-Fi. The cloud dashboard logs and displays the air quality level on a graph or gauge for the user to monitor remotely.

3. Circuit Diagram and Components

Components Used:

- ESP32 DevKit V1 Microcontroller

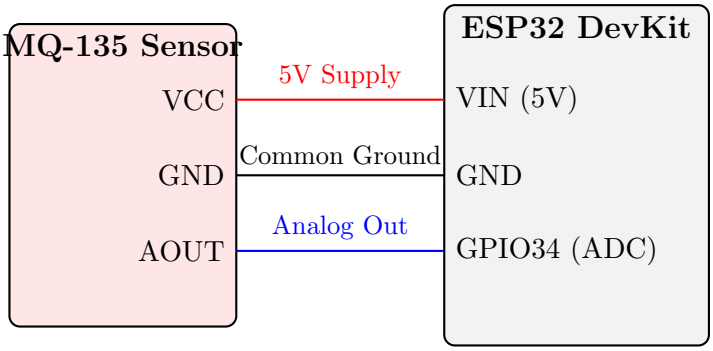


Figure 2: Circuit Diagram of Air Quality Monitoring System

- MQ-135 Gas Sensor
- Jumper Wires, Breadboard
- 5V USB Power Supply

Working Principle: The MQ-135 sensor measures the environmental gas concentration and outputs an analog voltage proportional to the gas levels. The ESP32 reads the analog voltage at GPIO34 (ADC) and converts it into a gas concentration reading. The ESP32 connects to Wi-Fi and the processed data is updated to the cloud dashboard in real time.

4. Code and Interfacing Logic

Interfacing Connections:

- MQ-135 Gas Sensor (AOUT) → GPIO34 (ESP32)
- MQ-135 Gas Sensor (VCC) → 5V (ESP32 VIN pin)
- MQ-135 Gas Sensor (GND) → ESP32 Common Ground

Code Description:

- Initializes cloud dashboard API and Wi-Fi credentials.
- Reads analog data from the MQ-135 gas sensor at periodic intervals.
- Converts raw ADC readings into meaningful air quality values.
- Sends data telemetry via HTTP/MQTT to the cloud platform (ThingSpeak/Blynk).
- If the computed value exceeds safe limits, alerts can be triggered remotely via the cloud dashboard notifications.

DOIT ESP32 DEVKIT V1

AirQualitySystem.ino | README.md

```
#include <ThingSpeak.h>

// ---------- PIN DEFINITIONS ----------
#define MQ135_PIN 34    // MQ-135 Analog Output Pin (GPIO34)

// ---------- THRESHOLD ----------
#define SENSOR_THRESHOLD 2000  // Example threshold for ADC reading (0-4095)

// ---------- WiFi & ThingSpeak ----------
const char* ssid = "GOPALAKRISHNAN";
const char* password = "4812561727";

unsigned long channelID = 3334454; // Replace with your ThingSpeak channel ID
const char* writeAPIKey = "XI72PMC80QWKIH4I"; // Replace with your ThingSpeak Write API key

WiFiClient client;
unsigned long lastThingSpeakUpdate = 0;

void setup() {
  Serial.begin(115200);

  // Initialize Pins
  pinMode(MQ135_PIN, INPUT);

  Serial.println("⏳ Air Quality Monitoring System Starting...");
  Serial.println("Waiting 2 minutes for sensor warm-up...");

  // Connect to WiFi
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n✅ WiFi Connected!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
```

Figure 3: Arduino Code

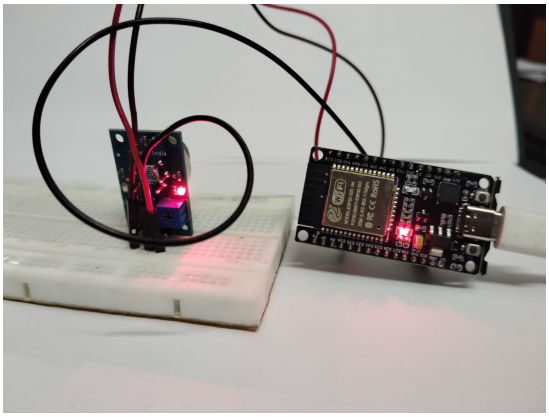
DOIT ESP32 DEVKIT V1

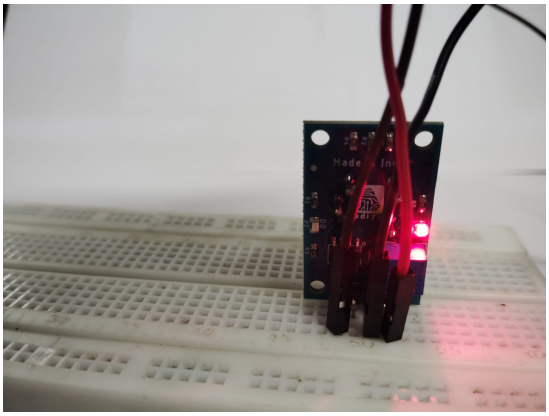
AirQualitySystem.ino | README.md

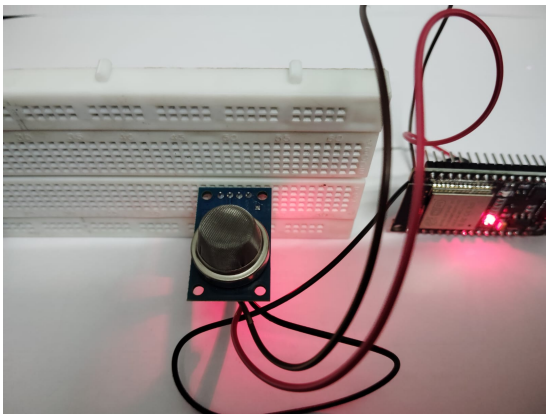
```
  ThingSpeak.begin(client);
}

void loop() {
  // ----- MQ-135 SENSOR READING -----
  int sensorValue = analogRead(MQ135_PIN);
  Serial.print("💨 Air Quality (Raw ADC Value): ");
  Serial.println(sensorValue);

  // Threshold Comparison
  if (sensorValue > SENSOR_THRESHOLD) {
    Serial.println("⚠️ Poor Air Quality Detected!");
  } else {
    Serial.println("✅ Air Quality is Good.");
  }
  Serial.println("----------------------------");

  // ----- ThingSpeak update every 15 sec -----
  if (millis() - lastThingSpeakUpdate > 15000) {
    lastThingSpeakUpdate = millis();

    if (WiFi.status() == WL_CONNECTED) {
      // Set the field value (Field 1)
      ThingSpeak.setField(1, sensorValue);

      // Write data to ThingSpeak
      int response = ThingSpeak.writeFields(channelID, writeAPIKey);
      if (response == 200) {
        Serial.println("✅ ThingSpeak Updated Successfully");
      } else {
        Serial.print("❌ ThingSpeak update failed, code: ");
        Serial.println(response);
      }
    } else {
      Serial.println("❌ WiFi not connected, cannot update ThingSpeak");
    }
  }
```

Figure 4: Arduino Code

5. Working Model







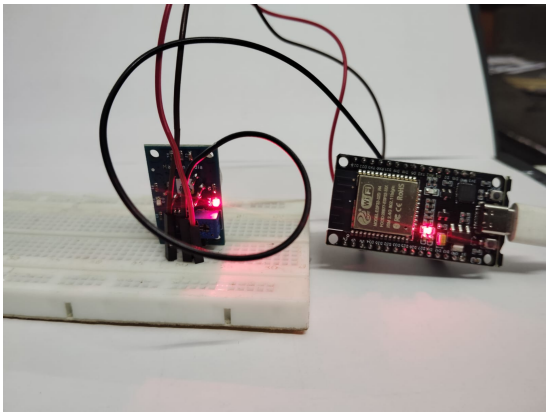


Figure 5: Working model of the Air Quality Monitoring System

6. System Output

The real-time data received from the sensor is processed by the ESP32 and successfully displayed on the output dashboard.

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0030,len:4744
load:0x40078000,len:15672
load:0x40080400,len:3152
entry 0x4008059c
Air Quality Monitoring System Starting...
Waiting 2 minutes for sensor warm-up...
Connecting to WiFi.......
WiFi Connected!
IP Address: 192.168.1.5
Air Quality (Raw ADC Value): 113
Air Quality is Good.
-----------------------------
Air Quality (Raw ADC Value): 98
Air Quality is Good.
-----------------------------
Air Quality (Raw ADC Value): 88
Air Quality is Good.
-----------------------------
Air Quality (Raw ADC Value): 86
Air Quality is Good.
-----------------------------
Air Quality (Raw ADC Value): 87
Air Quality is Good.
-----------------------------
```

Figure 6: Cloud Dashboard Output and Data Logs

Figure 7: Cloud Dashboard Output and Data Logs

7. Conclusion

The Air Quality Monitoring System measures harmful gas concentrations and enables remote tracking using IoT technology. The combination of the ESP32 microcontroller, MQ-135 sensor, and a cloud dashboard offers a cost-effective and scalable solution for smart environmental monitoring. The system continuously tracks pollution levels and provides users with real-time insights for evaluating and managing indoor and outdoor air quality efficiently.